COMP202
Complexity of Algorithms
Sorting

Chapter 6, Chapter 8 in CLRS.

Learning outcomes

1. Understand the sorting problem and its fundamental importance in algorithms.
2. Understand the limitations of comparison-based sorting algorithms.
3. Have a wide knowledge of different types of sorting algorithms available (such as Heap Sort, Counting Sort, Radix Sort) as well as their algorithmic complexity (i.e., running time).

Sorting

Sorting problem: Given a collection, C, of n elements (and a total ordering) arrange the elements of C into *non-decreasing* order, e.g.

45	3	67	1	5	16	105	8

1	3	5	8	16	45	67	105

Sorting (cont.)

Sorting is a fundamental algorithmic problem in computer science.

Many algorithms perform sorting (as a subroutine) during their execution. Hence, efficient sorting methods are crucial to achieving good algorithmic performance.

We will investigate various methods that we can use to sort items. Why several methods?

Sorting (cont.)

We may not always require a fully sorted list (for example), so some methods might be more appropriate depending upon the exact task at hand.

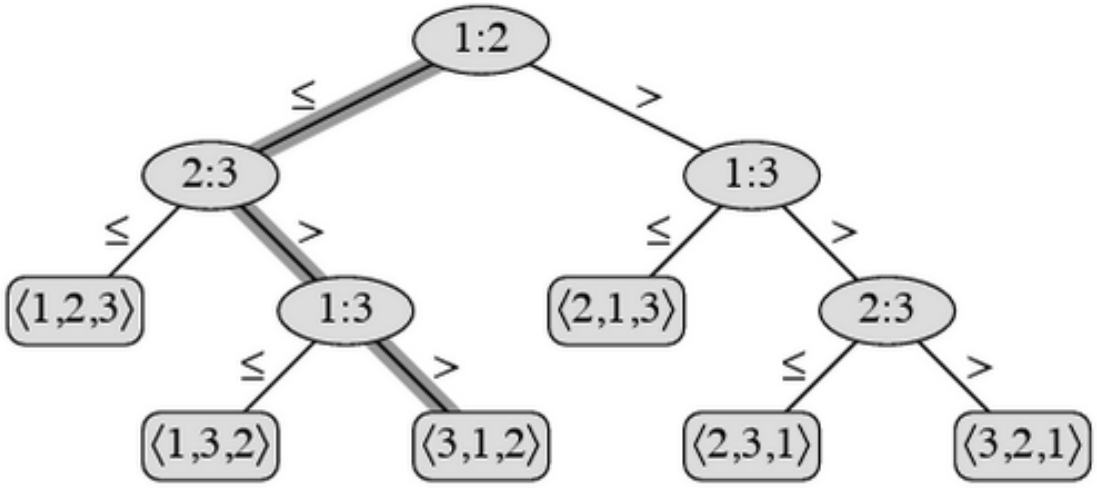
Sorting algorithms might be directly adaptable to perform additional tasks and directly provide solutions in this fashion.

Comparison-based sorting

We've already seen two sorting algorithms. These are MergeSort and Randomized QuickSort.

These are both *comparison-based* algorithms. Recall that MergeSort had a time complexity of $O(n \log n)$. We will first show that no comparison-based sorting technique can have a better running time.

Decision tree lower bound



The above is an example of a sorting algorithm on an input list of 3 elements. The notation $i : j$ means a_i is compared with a_j.

Internal nodes are unit cost, leaves correspond to the output of the algorithm on that path. The cost of the algorithm is at least the depth of the decision tree.

Total number of leaves must be at least 6 (more generally, at least $n!$. Why?). Thus, the depth must be at least $\log(n!) = \Omega(n \log n)$ (**see Stirling's approximation**).

Data Structures

- Algorithmic computations require data (information) upon which to operate.

The speed of algorithmic computations is (partially) dependent on efficient use of this data.

Data structures are specific methods for storing and accessing information.

We study different kinds of data structures as one kind may be more appropriate than another depending upon the *type* of data stored, and *how* we need to use it for a particular algorithm.

Data Structures (cont.)

Many modern high-level programming languages, such as C++, Java, and Perl, have implemented various data structures in some format.

For specialized data structures that aren't implemented in these programming languages, many can typically be constructed using the more general features of the programming language (e.g. pointers in C++, references in Perl, etc.).

Ordered Data

We often wish to store data that is ordered in some fashion (typically in numeric order, or alphabetical order, but there may be other ways of ordering it).

Arrays and lists are obvious structures that may be used to store this type of data.

Arrays aren't generally efficient to maintain data where we must add/delete items *and* maintain a sorted order. Linked lists may provide a better method in that case, but it is generally harder to *search for* an item in a list.

Priority Queues

A *Priority Queue* is a container of elements, each having an associated *key*.

Keys determine the *priority* used in picking elements to be removed.

A *priority Queue* (PQ) has these fundamental methods:

- *insertItem(k,e)*: insert element *e* having key *k* into PQ.
- *removeMax()*: remove maximum element.
- *maxElement()*: return maximum element.
- *maxKey()*: return key of maximum element.

(Of course, we could have priority queues that are based on maintaining the minimum elements.)

Priority Queue - Sorting

How can we use a priority queue to perform sorting on a set C?
Do this in two phases:

- *First phase*: *Put* elements of C into an initially empty priority queue, P, by a series of n *insertItem* operations.
- *Second phase*: *Extract* the elements from P in *non-increasing* order using a series of n *removeMax* operations.

PQ Sorting - Algorithm

PQ-SORT(C, P)

▷ Input: An n element sequence C and a priority queue P.
▷ Output: The sequence C sorted using the total order relation.

```
1  while C ≠ ∅
2      do
3          e ← C.REMOVEFIRST()
4          P.INSERTITEM(e, e)
5  while P ≠ ∅
6      do
7          e ← P.REMOVEMAX()
8          C.INSERTLAST(e)
```

Heap Data Structure

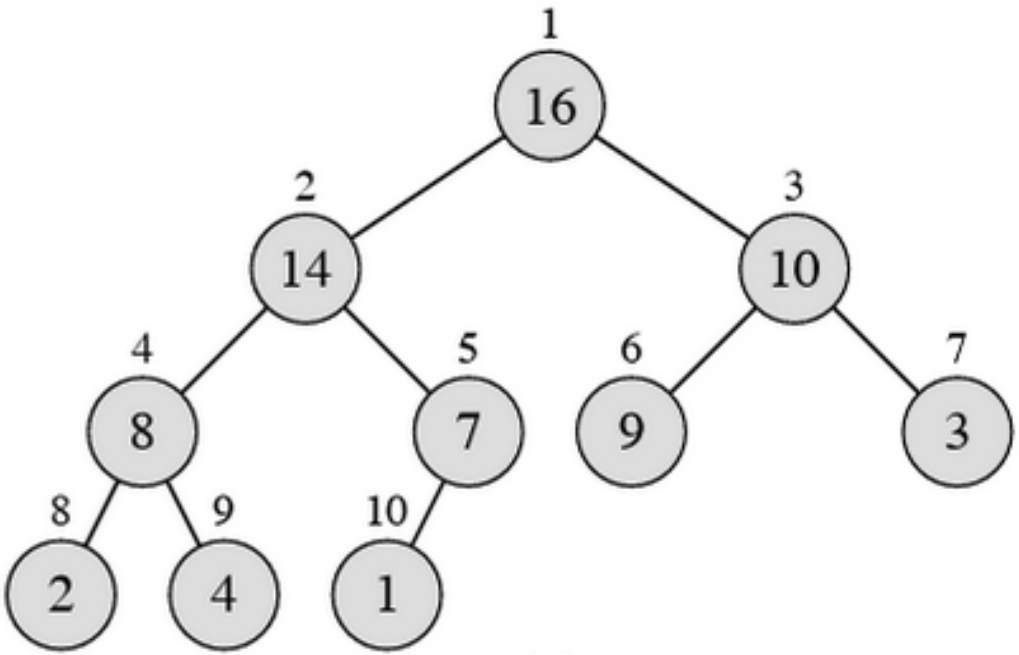
A *heap* is a realization of a Priority Queue that is *efficient* for both *insertions* and *deletions*.

A *heap* allows insertions and deletions to be performed in *logarithmic* time.

In a *heap* the *elements* and their *keys* are stored in an almost complete binary tree. Every level of the binary tree, except possibly the last one, will have the maximum number of children possible.

Heap-order property

- In a *heap* T, for every *node* v (excluding the *root*) the *key* at v is *less* than (or *equal* to) the *key* stored at its parent.



PQ/Heap implementation

An efficient realization of a heap can be achieved using an array for storing the elements.

So a heap can be implemented with the following:

- *heap*: A (nearly complete) binary tree T containing elements with *keys* satisfying the *heap-order property*, stored in an array.
- *last*: A *reference* to the *last used* node of T in this array representation.
- *comp*: A *comparator* function that defines the *total order relation* on *keys* and which is used to maintain the *maximum* (or minimum) element at the *root* of T.

Insertion in heaps

Inserting a new item into a heap begins by adding this element to the bottom of the tree in the position of the first unused, or empty, child.

Then, if necessary, this new element “bubbles” its way up the heap until the heap-order property is restored.

Deletion in a heap

Deletion in a heap consists of removing the minimum (or maximum) element (at the root) from the heap. Then the bottom, right-most element in the heap (the element at the end of the array that stores the heap) is moved to the root.

To restore the heap-order property, this item then “sinks” or bubbles down the heap.

An example of both of these operations will be (was) shown in the lecture.

Heap performance

Since a heap is an almost-complete binary tree, it stores n items in a tree of height $O(\log n)$. Thus we have the following summary of running times for operations that can be performed on a heap.

Operation	**time**
size, isEmpty	$O(1)$
maxElement, maxKey	$O(1)$
insertItem	$O(\log n)$
removeMax	$O(\log n)$

Heap-Sorting

Heap-sort: “Convert” input array into a heap. Perform n removeMax operations.

Theorem: The heap-sort algorithm sorts a sequence, S, of n comparable items in $O(n \log n)$ time, where

- Bottom-up construction of heap with n elements takes $O(n \log n)$ time, and
- Extraction of n elements (in increasing order) from the heap takes $O(n \log n)$ time.

Linear-time algorithms

We will now see some examples of linear-time ($O(n)$) sorting algorithms.

(Didn't we see an $\Omega(n \log n)$ lower bound earlier?)

The algorithms that we'll see will have the following 2 properties:

- They aren't comparison based, and
- They require a promise that the elements in the input aren't too large.

Counting sort

Counting sort assumes that each of the n input elements is an integer between 0 and k. When $k = O(n)$, counting sort runs in $O(n)$ time.

Input array is $A[1..n]$. Our output will be in $B[1..n]$, and we'll use $C[0..k]$ as temporary working storage.

Counting sort (cont.)

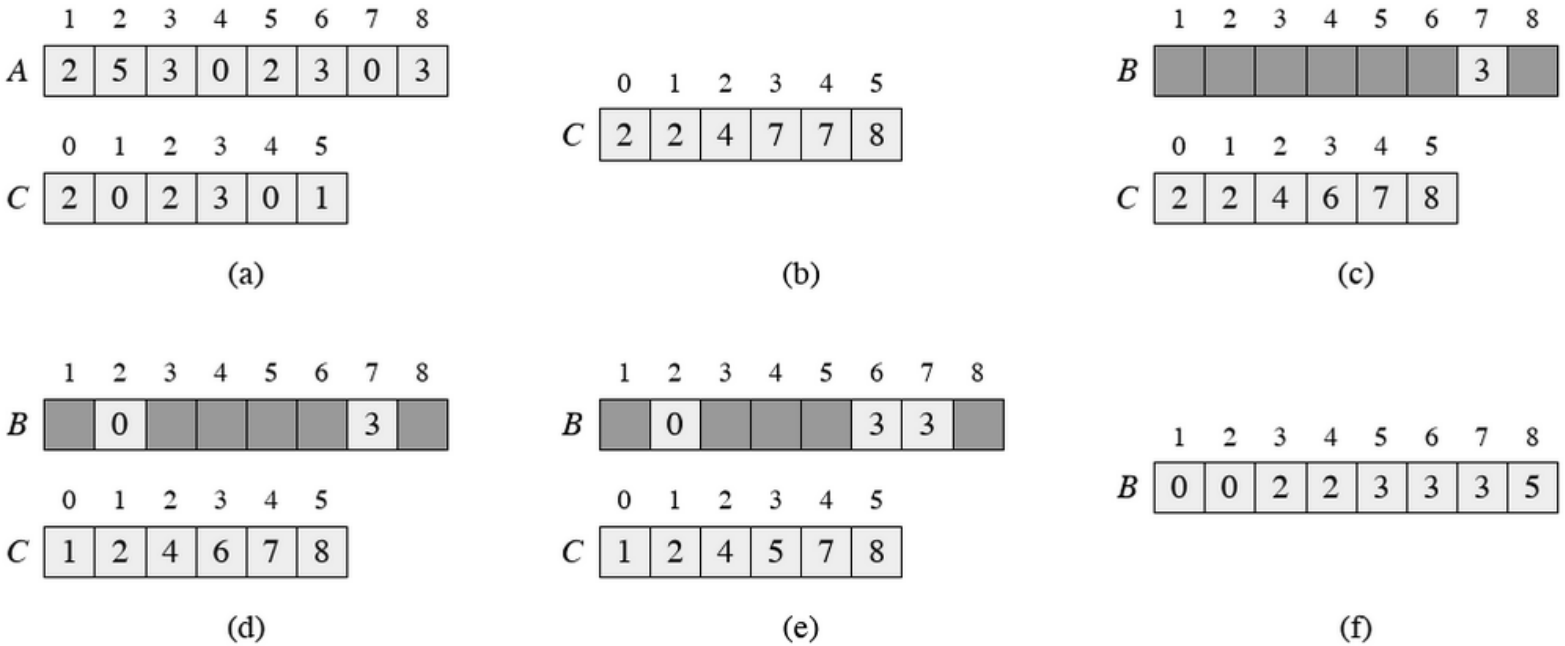
In one pass over A, fill C such that $C[k]$ contains the number of occurrences of k in A.

In another pass over C, update C such that $C[k]$ contains the number of elements *less than or equal to* k in A.

From the last to the first element of A, insert this element in the appropriate location in B and update C accordingly.

```
COUNTING-SORT(A, B, k)
 1  let C[0..k] be a new array
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to A.length
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  for j = A.length downto 1
11      B[C[A[j]]] = A[j]
12      C[A[j]] = C[A[j]] − 1
```

Example of counting sort



Stability

Loop invariant: before every iteration in the last **for** loop, $C[i]$ contains the list of numbers remaining in A that are at most i, for all $i \in \{0, 1, \ldots, k\}$. Thus, B is updated correctly (why are there no collisions in B?), so is C, and thus the loop invariant is maintained.

Counting sort is **stable**: numbers with the same value appear in the same order in the output array as they appear in the input array.

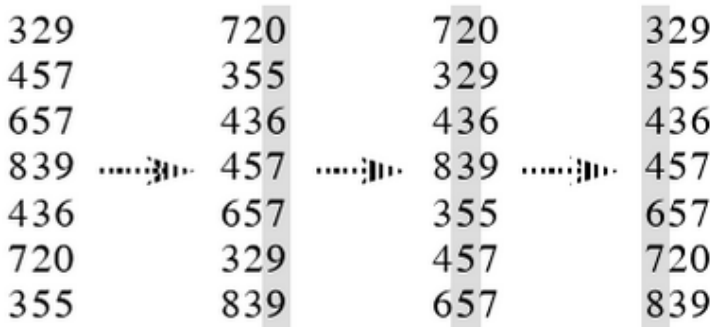
Stability is a practically-useful concept because there may be big chunks of data also associated with each number, and we may want the data's order to always be preserved.

More importantly for us, in order for radix sort (coming up) to work correctly, counting sort must be stable.

Radix sort

Used to be used by card-sorting machines that sorted punch cards containing data.

Counter-intuitive: First sort by the *least significant* digit. Next sort by second-least significant digit, and so on. After running through each digit, the input is (remarkably) sorted.



Analysis of radix sort

Running time: $O(n)$ per digit. Thus, if all inputs are d digits, radix sort runs in time $O(dn)$.

Correctness: Can be shown by induction (on which digit the algorithm looks at). Crucially requires each column to be sorted using a stable sort.